

Software Architecture Document

Introduction

This Software Architecture Document (SAD) outlines the architectural framework and high level design for “The Greatest Game of Minesweeper”, a pygame version of Minesweeper developed as part of our EECS 581 Coursework at the University of Kansas. This blueprint helps ensure that the development aligns with the Software Requirements Specifications (SRS), guiding the team towards successful implementation. This is the updated documentation with information about the new added features included

Purpose

This document delivers a comprehensive architectural overview of The Greatest Game of Minesweeper, capturing key decisions and presenting them via various architectural views. It is designed for the project team, to guide implementation, and for stakeholders seeking detailed insights into the software's capabilities. Our group build upon the previous groups project to add several new features including updating the documentation and the work that followed.

Scope

The scope of this document encompasses the complete architecture of theThe Greatest Game of Minesweeper, including Graphics Rendering, Mouse Input, Logic, Data Structures for API interactions and a Game Manager that controls the game's state.

Definitions, Acronyms, and Abbreviations

- SRS: Software Requirements Specification
- API: Application Programming Interface
- UI: User Interface
- pygame: A python framework that offers function calls for features often used in game development
- GUI: Graphical User Interface
- AI: Artificial Intelligence

References

- EECS 348 Software Engineering Course Syllabus, Professor Hossein Saiedian, University of Kansas, Spring Semester 2024.
- EECS 581Project 1: Minesweeper System Development Specifications, Professor Hossein Saiedian, University of Kansas, Fall Semester 2025.
- Group 16's github repository and documentation.

Overview

This document covers our software design architecture. It will touch on various items such as size, interface, and structure. It covers:

- Architectural Representation: Outlines views and models representing the system.
- Architectural Goals and Constraints: Details architectural requirements and objectives.
- Logical View: Describes system decomposition into packages and classes.
- Interface Description: Provides high-level details of key interfaces.
- Development Description: Overview of the hours we worked and our development process
- Quality: Explains how the architecture supports quality attributes such as extensibility and Reliability.

Architectural Representation

The software architecture for The Greatest Game of Minesweeper system encompasses multiple views that collectively provide a comprehensive understanding of the system's structure and behavior. Each view focuses on specific aspects of the architecture, facilitating communication among stakeholders and guiding the development and evolution of the system.

Overview:

Description: Provides a high-level overview of the system architecture, highlighting its key components and their interactions.

Model Elements:

- Main components of the system (e.g., user interface, input handling, data structures).
- High-level communication pathways between components.
- Deployment environment (e.g., computer details, python and pygame version).

Logical View:

Description: Describes the logical organization of the system, focusing on the decomposition of functionality into modules or layers.

Model Elements:

- Modules representing major functional areas (e.g., board.py, renderer.py).
- Relationships and dependencies between modules.
- Interfaces exposed by each module for interaction with other parts of the system.

Process View:

Description: Shows the dynamic behavior of the system, illustrating how processes and threads interact to fulfill user input on the Minesweeper board.

Model Elements:

- Processes representing user interactions (e.g., clicking on the board, showing mines/flags).
- Communication pathways between processes (e.g., data structures, imported packages).

Development View:

Description: Focuses on the organization of the development effort, including the structure of source code, version control, and development environments.

Model Elements:

- Source code repositories and their organization (e.g., repositories for frontend, backend, testing).
- Development environments (e.g., local development setups, GitHub Projects)

Architectural Goals and Constraints

- Developmental tools: We will utilize Visual Studio Code for its versatility, Git and GitHub for version control and PyCharm / VSCode with the Python Extension, ensuring a robust development environment.
 - Design: Object Oriented Programming - to facilitate clear modularity and extensibility of the software components.
 - Usability: Develop an intuitive user interface and solid technical backend to enhance gameplay experience.
 - Reuse: Architect the codebase to promote code reuse and modularity for easier maintenance and scalability.
 - Off-the-shelf product: Integrate third-party APIs or libraries for features such as content recommendation
 - Design and implementation strategy: We plan to follow a waterfall development methodology, to ensure phases and requirements are met in a structured progression.

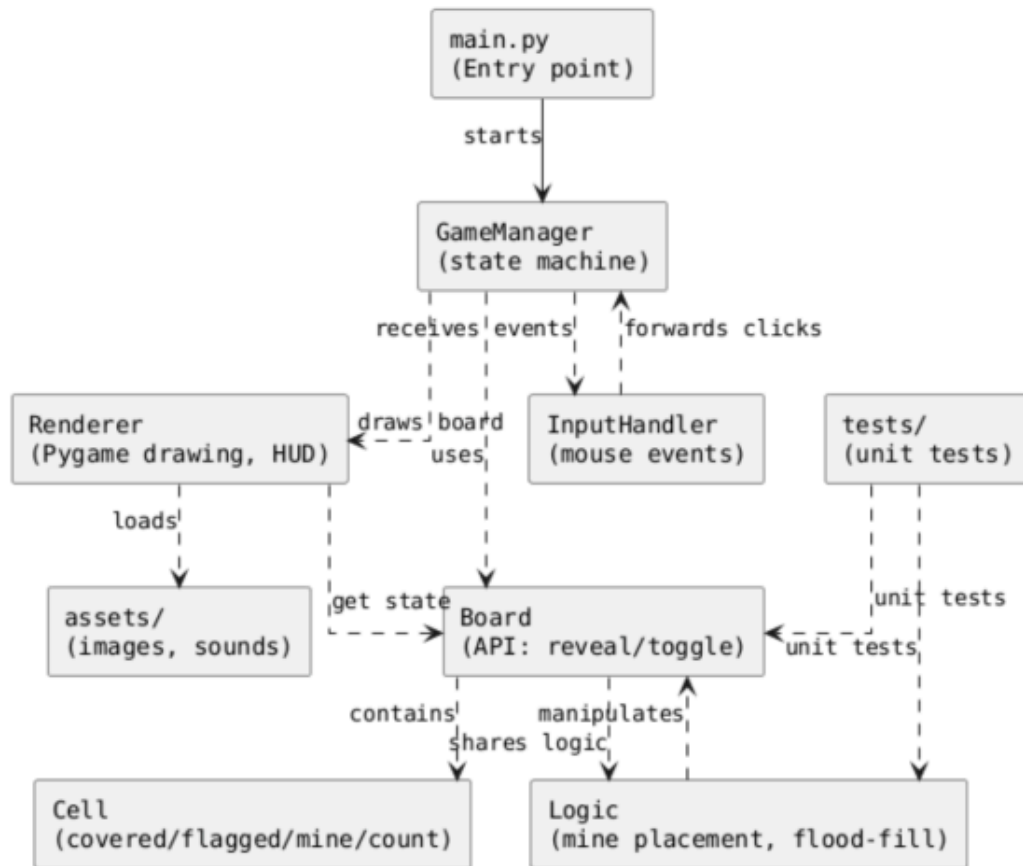
Logical Overview

The overall design of the Minesweeper program is divided into five major subsystems that interact to deliver the game's functionality. These are the

- Game Manager: acts as the central controller, orchestrating interactions between components, managing states (PLAYING, WIN, LOSS), and handling the game loop
- Board & Logic: both are responsible for mine placement, recursive uncovering (flood fill), and win/loss detection
- Data Structures (Cells/Board Classes): define the grid of Cells and Board API, storing essential game state information (mines, flags, covered/uncovered)
- User Input Handler: processes mouse and keyboard inputs, mapping them to board actions (reveal cell, toggle flag, restart)
- Renderer: dynamically draws the board and UI using Pygame, showing cell states, flags, mines, counters, and banners.

Architecturally Significant Design

The Greatest Game of Minesweeper



5. Interface Description

The Project called “The Greatest Game of Minesweeper” interface will be built with Pygame, with a grid based design as per the EECS

581 project requirements:

- Valid Inputs:
 - Left click: Reveal a cell
 - Right click: Place/remove a flag
 - R Key on keyboard: Reset the board for a new game
 - Esc Key: Exit game
- Outputs:
 - Rendering of board updates (reveals, flags, mines)
 - Game status text (“Playing”, “Victory!”, “Game Over”)
- Data Storage:
 - Game state (board + cells) is stored in-memory within the Board object
 - No external persistence is required
- New GUI
 - No longer need command line integration
 - Added a refresh transition between the old and new game
 - Screen choices for mines and size with